

Praxisarbeit – DBWE.TA1A.PA

HausPlan – hausplan.nuscnet.ch

von Nury Schmed

Fach	Datenbanken und Webentwicklung (DBWE)
Lehrgang	HF Informatik, 3. Studienjahr, ipso! Bildung
Webanwendung	https://hausplan.nuscnet.ch
Quellcode	https://github.com/ulangtaun/hausplan
Stand	09.10.2026

Inhaltsverzeichnis

Management Summary	3
1 Einleitung	4
1.1 Ausgangslage.....	4
1.2 Ziel und Abgrenzung.....	4
1.3 Vorgehen.....	4
2 Anforderungen	5
2.1 Anforderungen aus der Aufgabenstellung.....	5
2.2 Funktionale Anforderungen.....	5
2.3 Qualitätsanforderungen.....	6
3 Benutzerhandbuch	7
3.1 Zugang.....	7
3.2 Registrieren und Anmelden.....	7
3.3 Haushalte.....	7
3.4 Projekt-Board.....	8
3.5 Projekt bewerten und bearbeiten.....	9
3.6 Planung.....	9
3.7 Einstellungen.....	10
3.8 Profil und API-Token.....	11
4 REST-API	12
4.1 Zugriff und Authentifizierung.....	12
4.2 Endpunkte.....	12
4.3 Beispiele.....	12
4.4 Fehlercodes.....	13
5 Architektur	14
5.1 Überblick und Code-Struktur.....	14
5.2 Datenmodell.....	14
5.3 Geschäftslogik.....	16
5.4 Wichtigste Abläufe.....	17
5.5 Zusätzliche Technologien.....	19
5.6 Bereitstellung.....	20
5.7 Sicherheit.....	21
6 Reflexion	22
6.1 Wartbarkeit.....	22
6.2 Skalierbarkeit.....	22
6.3 Verfügbarkeit.....	22
6.4 Erkenntnisse aus der Umsetzung.....	22
7 Tests	24
7.1 Teststrategie.....	24
7.2 Testprotokoll.....	24
8 Fazit und Ausblick	26
Quellenverzeichnis	27
Abbildungsverzeichnis	28
Anhang	29
A Zugriffsinformationen.....	29
B Installation und lokaler Start.....	29
C Eingesetzte Hilfsmittel.....	29

Management Summary

Vorhaben. Im Rahmen der Praxisarbeit im Fach Datenbanken und Webentwicklung wurde die Webanwendung **HausPlan** entwickelt und im Internet bereitgestellt. HausPlan hilft einem Haushalt, Ideen und Projekte für zuhause zu sammeln, gemeinsam zu bewerten und realistisch zu planen: Welche Projekte bringen am meisten, sind wir uns einig, in welcher Reihenfolge gehen wir vor und was liegt im Quartalsbudget drin?

Ziele. Erfüllt werden sollten alle Anforderungen der Aufgabenstellung: eine interaktive Weboberfläche mit Benutzerkonten, eine relationale Datenbank, eine eigene Geschäftslogik, ein RESTful Web-API mit Authentifizierung ohne Browser sowie der Betrieb im Internet während mindestens vier Wochen.

Ergebnis. Die Anwendung ist unter <https://hausplan.nuscnet.ch> öffentlich erreichbar, der kommentierte Quellcode liegt auf GitHub (github.com/ulangtaun/hausplan). Die selbst entworfene Geschäftslogik umfasst eine gewichtete Nutzwertanalyse mit Konsens-Erkennung, einen Status-Workflow mit Regeln, Abhängigkeiten mit Zyklenerkennung und empfohlener Reihenfolge sowie einen Budget-Vorschlag. 30 automatisierte Tests laufen erfolgreich, alle 12 Testfälle des Testprotokolls sind bestanden.

Infrastruktur. Die App läuft auf dem eigenen Proxmox-Cluster in zwei getrennten Linux-Containern (Debian 13): einer für die Anwendung (Flask, Unicorn, Nginx), einer für die Datenbank (PostgreSQL 17). Veröffentlicht wird sie über einen **Cloudflare Tunnel**: Am Heimrouter ist kein Port geöffnet, die private IP-Adresse bleibt verborgen und Cloudflare stellt das TLS-Zertifikat bereit. Es fallen keine Cloud-Kosten an, nur die Gebühr für die Domain.

Wichtigster Mehrwert	Entscheidungen im Haushalt werden fair und nachvollziehbar: Alle bewerten gleichberechtigt, Uneinigkeit wird sichtbar, und der Plan berücksichtigt Budget, Zeit und Abhängigkeiten.
Grösstes Risiko	Die Verfügbarkeit hängt von der Heiminfrastruktur ab (Strom, Internetanschluss, ein einzelner Proxmox-Node). Gegenmassnahmen: automatischer Start der Container, tägliche Backups, Cloudflare-Tunnel ohne feste IP, Verzicht auf Updates während der Korrekturzeit.
Das sollte das Management wissen	Für einen produktiven Einsatz mit vielen Nutzern wären Hochverfügbarkeit (gemeinsamer Speicher oder Cloud-Hosting), ein externes Monitoring und eine Datenschutzprüfung nötig. Die Architektur erlaubt diesen Schritt ohne Umbau der Anwendung.

1 Einleitung

1.1 Ausgangslage

In einem Haushalt entstehen laufend Ideen: die Wand streichen, ein Hochbeet bauen, den Keller entrümpeln, die Heizung smart machen. Oft fehlt der Überblick, was wirklich wichtig ist, und die Prioritäten der Mitbewohnenden unterscheiden sich. Projekte werden begonnen, bevor die Voraussetzungen erfüllt sind, oder sprengen das Budget. Heute werden solche Listen meist auf Zetteln, in Chats oder Tabellen geführt, die keine gemeinsame Bewertung und keine Planung unterstützen.

1.2 Ziel und Abgrenzung

Ziel ist eine Webanwendung, mit der ein Haushalt Projekte gemeinsam erfasst, nach eigenen Kriterien bewertet und mit Rücksicht auf Budget, verfügbare Zeit und Abhängigkeiten plant. Die Anwendung erfüllt die Anforderungen der Aufgabenstellung (Kapitel 2) und wird für die Korrekturzeit im Internet bereitgestellt.

Nicht Teil dieser Arbeit sind: Datei-Uploads (z. B. Fotos), E-Mail-Benachrichtigungen, Kalender-Integration, eine native Mobile-App sowie Mehrsprachigkeit. Diese Punkte sind im Ausblick (Kapitel 8) aufgeführt.

1.3 Vorgehen

1. **Ideenfindung und Bewertung:** Mehrere Ideen wurden gegen die Kriterien der Aufgabenstellung geprüft (Geschäftslogik, Datenmodell, Nutzen des API, Umfang, Testbarkeit). Gewählt wurde das Projekt-Board, ergänzt um Funktionen, die echte Geschäftslogik liefern.
2. **Entwurf:** Datenmodell, Status-Workflow und Algorithmen wurden vor der Umsetzung festgelegt.
3. **Umsetzung:** Flask-Anwendung nach der Struktur aus dem Unterricht (Flask Mega-Tutorial), Geschäftslogik in eigenen Modulen, automatisierte Tests parallel zur Entwicklung.
4. **Bereitstellung:** Einrichtung von zwei LXC-Containern auf Proxmox mit Skripten, Veröffentlichung über Cloudflare Tunnel.
5. **Test und Dokumentation:** Automatisierte und manuelle Tests, Testprotokoll und diese Dokumentation.

2 Anforderungen

2.1 Anforderungen aus der Aufgabenstellung

ID	Anforderung	Umsetzung in HausPlan	Erfüllt
A1	Interaktive Weboberfläche für Eingaben, Aktionen und Ergebnisse	Board, Projektdetail mit Bewertung und Statuswechsel, Planung, Einstellungen (Kapitel 3)	Ja
A2	Benutzerkonten mit Passwort, Registrierung mit eindeutigem Benutzernamen und E-Mail	Flask-Login, Passwort-Hash, Prüfung auf Eindeutigkeit (auch Gross-/Kleinschreibung)	Ja
A3	Anwendungsspezifische Daten in relationaler Datenbank	PostgreSQL 17 mit 7 Tabellen (Kapitel 5.2)	Ja
A4	Eigene Geschäftslogik	Nutzwertanalyse, Konsens, Workflow, Abhängigkeiten, Budget-Vorschlag (Kapitel 5.3)	Ja
A5	RESTful Web-API, mindestens lesend, für spezifische Inhalte; Authentifizierung ohne Browser	11 Endpunkte, Token-Authentifizierung mit Flask-HTTPAuth (Kapitel 4)	Ja
A6	Technologien: MySQL/MariaDB/PostgreSQL, Python ≥ 3.9 , Flask, Gunicorn (mit Nginx)	PostgreSQL 17, Python 3.13, Flask 3.1, Gunicorn 26 hinter Nginx 1.26	Ja
A7	Bereitstellung im Internet, mindestens 4 Wochen	https://hausplan.nuscnet.ch über Cloudflare Tunnel (Kapitel 5.6)	Ja
A8	Quellcode auf GitHub, kommentiert, Quellen gekennzeichnet	github.com/ulangtaun/hausplan , Quellen in Kommentaren	Ja
A9	Tests dokumentiert, Testprotokoll	30 automatisierte Tests, Testprotokoll (Kapitel 7)	Ja

Tabelle 1: Anforderungen aus der Aufgabenstellung und ihre Umsetzung

2.2 Funktionale Anforderungen

ID	Funktion
F1	Benutzer können Haushalte erstellen und andere Benutzer über den Benutzernamen als Mitglieder hinzufügen. Es gibt die Rollen Besitzer und Mitglied.
F2	Mitglieder erfassen Projekte mit Titel, Beschreibung, geschätzten Kosten (CHF) und Stunden.
F3	Besitzer definieren Bewertungskriterien mit Gewicht (1–5) und Richtung (Nutzen- oder Kosten-Kriterium).
F4	Jedes Mitglied bewertet ein Projekt pro Kriterium mit 1–5. Die App berechnet einen gemeinsamen Score von 0–100.
F5	Liegen die Bewertungen weit auseinander, wird das Projekt mit «Diskussionsbedarf» markiert.
F6	Projekte durchlaufen einen Status-Workflow mit Regeln; der Übergang «Idee → Bewertet» geschieht automatisch.
F7	Projekte können voneinander abhängen. Zirkuläre Abhängigkeiten werden abgelehnt, blockierte Projekte können nicht gestartet werden.
F8	Die App schlägt eine Reihenfolge vor, die Abhängigkeiten und Score berücksichtigt.

ID	Funktion
F9	Die App schlägt vor, welche Projekte ins Quartalsbudget und die verfügbare Zeit passen, und warnt bei Überschreitung.
F10	Beim Abschluss werden Ist-Kosten und Ist-Stunden erfasst; die App zeigt die durchschnittliche Schätzabweichung.

Tabelle 2: Selbst definierte funktionale Anforderungen

2.3 Qualitätsanforderungen

- **Sicherheit:** Passwörter nur als Hash gespeichert, CSRF-Schutz für alle Formulare, Zugriff nur auf Daten eigener Haushalte (Web und API), Datenbank nur vom App-Container erreichbar.
- **Wartbarkeit:** Geschäftslogik getrennt von Web und API, versioniertes Datenbankschema (Alembic), automatisierte Tests.
- **Verfügbarkeit:** Automatischer Neustart von Diensten und Containern, tägliche Backups, Erreichbarkeit ohne offene Ports.
- **Bedienbarkeit:** Responsive Oberfläche (Bootstrap 5), verständliche Fehlermeldungen, gesperrte Aktionen zeigen den Grund an.

3 Benutzerhandbuch

3.1 Zugang

Adresse	https://hausplan.nuscnet.ch (HTTPS, Port 443)
Testkonto 1	Benutzername examiner, Passwort 0kto2026 – Besitzer der «Demo-Wohnung» mit Beispieldaten
Testkonto 2	Benutzername mitbewohner, Passwort 0kto2026 – Mitglied der «Demo-Wohnung» und Besitzer des Haushalts «WG des Bruders», auf den examiner keinen Zugriff hat
Eigenes Konto	Über «Registrieren» kann jederzeit ein neues Konto angelegt werden.

Tabelle 3: Zugangsdaten für die Prüfung

3.2 Registrieren und Anmelden

Ohne Anmeldung ist nur die Hilfeseite zugänglich. Bei der Registrierung werden Benutzername, E-Mail-Adresse und Passwort (mindestens 8 Zeichen) verlangt. Benutzername und E-Mail-Adresse müssen eindeutig sein; bei einem Konflikt erscheint eine Fehlermeldung direkt beim Feld.

Jetzt registrieren'. At the very bottom of the page, there is small text: 'HausPlan · Praxisarbeit DBWE · Flask & PostgreSQL'." data-bbox="208 374 779 562"/>

Abbildung 1: Anmeldeseite

3.3 Haushalte

Nach der Anmeldung zeigt die Startseite alle Haushalte, in denen die Person Mitglied ist, mit einer Übersicht der Projekte pro Status. Rechts kann ein neuer Haushalt mit Budget und verfügbaren Stunden pro Quartal angelegt werden. Wer einen Haushalt erstellt, wird Besitzer; der Haushalt erhält vier Standard-Kriterien (Nutzen, Dringlichkeit, Freude, Aufwand).

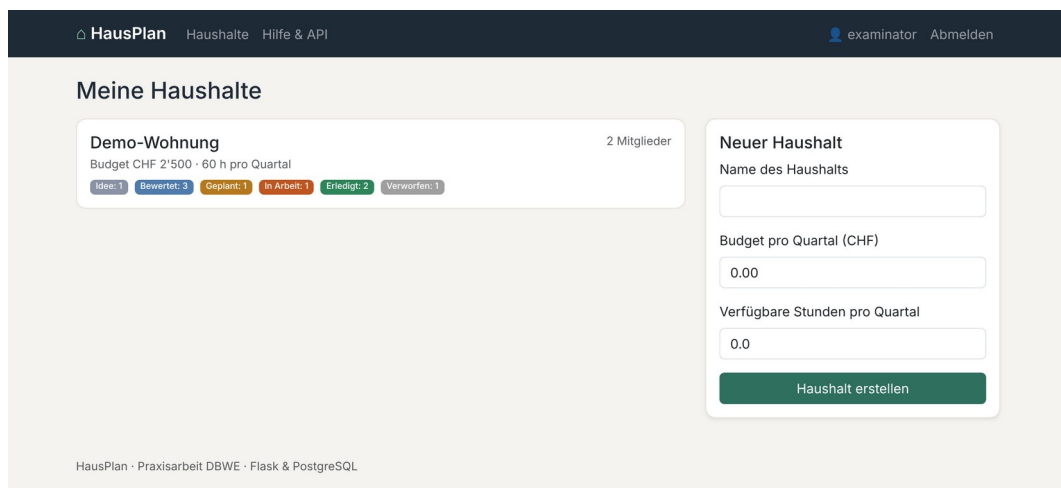


Abbildung 2: Übersicht «Meine Haushalte»

3.4 Projekt-Board

Das Board zeigt die offenen Projekte in Spalten nach Status, innerhalb einer Spalte nach Score sortiert. Oben stehen Budget, Zeit und der aktuelle Planungsvorschlag. Mit «+ Neue Idee» wird ein Projekt erfasst.

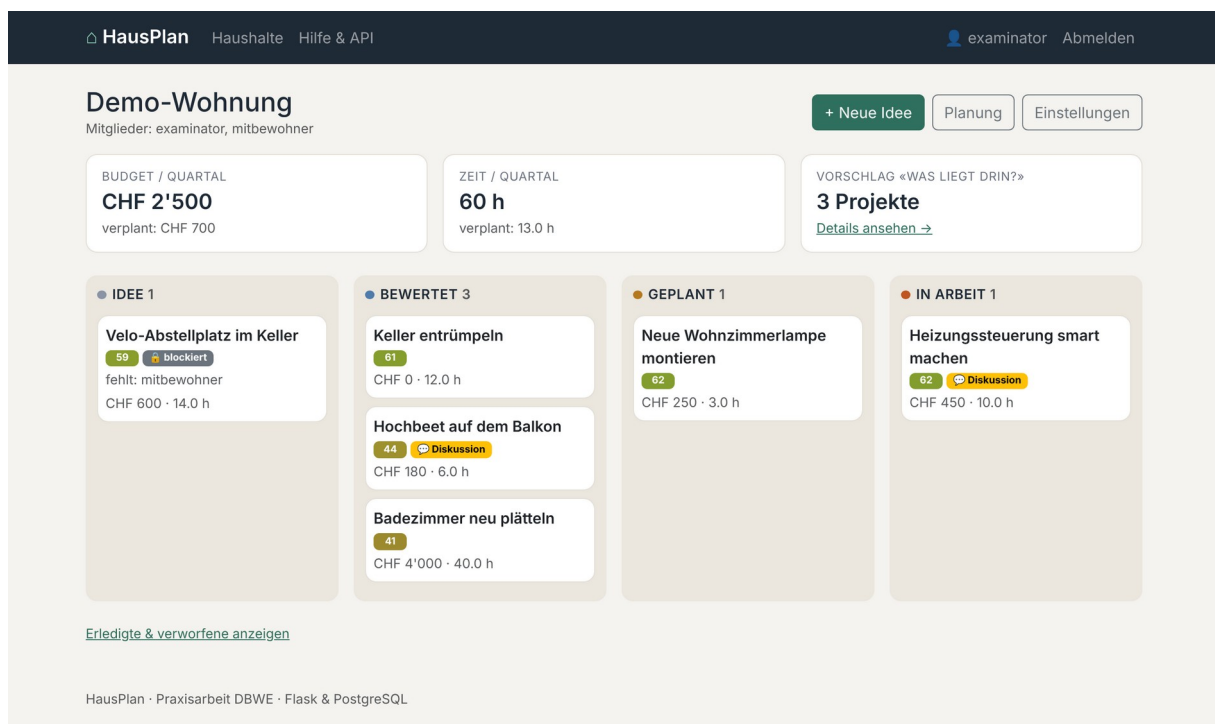


Abbildung 3: Projekt-Board der Demo-Wohnung

Symbol	Bedeutung
Farbige Zahl	Gemeinsamer Score 0–100 (rot = tief, grün = hoch)
🗨 Diskussion	Die Mitglieder sind sich bei diesem Projekt deutlich uneinig
🔒 blockiert	Ein Projekt, von dem dieses abhängt, ist noch nicht erledigt
fehlt: Name	Diese Person hat das Projekt noch nicht bewertet

Tabelle 4: Symbole auf dem Board

3.5 Projekt bewerten und bearbeiten

In der Projektansicht bewertet jedes Mitglied das Projekt pro Kriterium mit 1–5 und speichert. Rechts stehen die Einzelbewertungen und der gemeinsame Score. Liegen die Bewertungen weit auseinander, nennt die App die Gründe. Haben alle bewertet, wechselt das Projekt automatisch auf «Bewertet».

Unter «Abhängigkeiten» wird festgelegt, auf welche Projekte gewartet werden muss. Im Bereich «Status» stehen die erlaubten nächsten Schritte (Einplanen, Starten, Abschliessen, Verwerfen ...). Ist ein Schritt nicht erlaubt, ist der Button deaktiviert und der Grund steht darunter, z. B. «Blockiert durch offene Abhängigkeiten». Beim Abschliessen werden die effektiven Kosten und Stunden erfasst.

The screenshot shows the 'Hochbeet auf dem Balkon' project page in the HausPlan app. The page is divided into several sections:

- Header:** HausPlan logo, navigation links (Haushalte, Hilfe & API), user 'examinator', and 'Abmelden' button.
- Project Info:** 'Demo-Wohnung' link, project title 'Hochbeet auf dem Balkon', 'Bewertet' status with a score of 44, and a 'Diskussion' button. Description: 'Lärchenholz, ca. 120 x 60 cm, für Kräuter und Salat.' Buttons: 'Bearbeiten', 'Löschen'.
- Meine Bewertung:** Evaluation criteria with 1-5 scales:
 - Nutzen (Gewicht 3):** 'Wie sehr verbessert das Projekt unseren Alltag?' (Scale: 1, 2, 3, 4, 5)
 - Dringlichkeit (Gewicht 2):** 'Wie dringend muss es gemacht werden?' (Scale: 1, 2, 3, 4, 5)
 - Freude (Gewicht 1):** 'Wie viel Spass macht uns das Projekt?' (Scale: 1, 2, 3, 4, 5)
 - Aufwand (Gewicht 2, Kosten-Kriterium):** 'Wie viel Zeit, Geld und Nerven kostet es? (5 = sehr viel)' (Scale: 1, 2, 3, 4, 5)'Bewertung speichern' button.
- Status:** 'Einplanen' (active) and 'Verwerfen' (disabled) buttons.
- Bewertungen im Haushalt:** Table showing scores for 'examinator' (69 / 100) and 'mitbewohner' (19 / 100), with a 'Gemeinsamer Score' of 43.8.
- Diskussionsbedarf:** Yellow box listing reasons for discussion:
 - Gesamtbewertungen liegen 50 Punkte auseinander.
 - Uneinig bei «Nutzen» (2 vs. 5).
 - Uneinig bei «Freude» (2 vs. 5).
- Aufwand:** 'Geschätzt' CHF 180 · 6.0 h, 'Erfasst von' examiner, 'Erfasst am' 09.10.2026.
- Abhängigkeiten:** 'Dieses Projekt kann erst starten, wenn diese Projekte erledigt sind.' List: 'Keine', 'Wohnzimmerwand streichen (Erledigt)' (dropdown), 'Hängt ab von' button.
- Footer:** HausPlan · Praxisarbeit DBWE · Flask & PostgreSQL

Abbildung 4: Projektansicht mit Bewertung, Diskussionsbedarf und Status

3.6 Planung

Die Seite «Planung» zeigt drei Auswertungen: den Vorschlag «**Was liegt drin?**» (welche bewerteten Projekte in Budget und Zeit des Quartals passen, mit Begründung für nicht gewählte Projekte), die **empfohlene Reihenfolge** unter Berücksichtigung der Abhängigkeiten und die **Schätzgenauigkeit** der abgeschlossenen Projekte.

HausPlan
Haushalte
Hilfe & API

examinator
Abmelden

[← Demo-Wohnung](#)

Planung

Was liegt drin? Vorschlag für dieses Quartal

Auswahl mit dem höchsten Nutzen pro eingesetztem Franken und Stunde – unter Berücksichtigung von Budget, Zeit und Abhängigkeiten. Laufende Projekte sind bereits eingerechnet.

Budget: CHF 880 von CHF 2'500 · Zeit: 31.0 von 60.0 h

Projekt	Score	Kosten	Stunden
Heizungssteuerung smart machen In Arbeit		CHF 450	10.0
✓ Neue Wohnzimmerlampe montieren Geplant	62	CHF 250	3.0
✓ Keller entrümpeln Bewertet	61	CHF 0	12.0
✓ Hochbeet auf dem Balkon Bewertet	44	CHF 180	6.0

Nicht enthalten

- [Badezimmer neu plätteln](#) – Budget reicht nicht (braucht CHF 4000, frei 1620)
- [Velo-Abstellplatz im Keller](#) – Noch nicht von allen bewertet

Empfohlene Reihenfolge

Berücksichtigt Abhängigkeiten; bei freier Wahl kommt der höhere Score zuerst.

- [1. Heizungssteuerung smart machen](#) (62)
- [2. Neue Wohnzimmerlampe montieren](#) (62)
- [3. Keller entrümpeln](#) (61)
- [4. Velo-Abstellplatz im Keller](#) (59)
nach: Keller entrümpeln
- [5. Hochbeet auf dem Balkon](#) (44)
- [6. Badezimmer neu plätteln](#) (41)

Schätzgenauigkeit

Kosten: Ihr habt im Schnitt um 7 % unterschätzt.
Zeit: Ihr habt im Schnitt um 31 % unterschätzt.
Basis: 2 abgeschlossene Projekte.

HausPlan · Praxisarbeit DBWE · Flask & PostgreSQL

Abbildung 5: Planung mit Budget-Vorschlag, Reihenfolge und Schätzgenauigkeit

3.7 Einstellungen

Nur Besitzer sehen die Einstellungen. Dort werden Name, Budget und Stunden des Haushalts gepflegt, Mitglieder hinzugefügt oder entfernt, Rollen vergeben und die Bewertungskriterien angepasst. Eine Änderung des Gewichts wirkt sofort auf alle Scores. Der letzte Besitzer kann nicht entfernt werden.

HausPlan

Haushalte

Hilfe & API

examinator

Abmelden

← Demo-Wohnung

Einstellungen

Haushalt & Planungsrahmen

Name des Haushalts

Demo-Wohnung

Budget pro Quartal (CHF)

2500.00

Verfügbare Stunden pro Quartal

60.0

Speichern

Mitglieder

examinator

Besitzer

zu Mitglied

entfernen

mitbewohner

zu Besitzer

entfernen

Benutzername

Hinzufügen

Bewertungskriterien

Jedes Mitglied bewertet jedes Projekt pro Kriterium mit 1-5. Das Gewicht bestimmt, wie stark ein Kriterium in den Score einfließt. Bei Kosten-Kriterien ist ein hoher Wert schlecht.

Kriterium	Richtung	Gewicht	
Nutzen Wie sehr verbessert das Projekt unseren Alltag?	Nutzen ↑	3	löschen
Dringlichkeit Wie dringend muss es gemacht werden?	Nutzen ↑	2	löschen
Freude Wie viel Spass macht uns das Projekt?	Nutzen ↑	1	löschen
Aufwand Wie viel Zeit, Geld und Nerven kostet es? (5 = sehr viel)	Kosten ↓	2	löschen

Kriterium hinzufügen

Kriterium

Gewicht

2

Hilfextext

Richtung

Je höher, desto besser (z. B. Nutzen)

Kriterium hinzufügen

HausPlan · Praxisarbeit DBWE · Flask & PostgreSQL

3.8 Profil und API-Token

Unter «Profil & API» kann ein API-Token angezeigt, erstellt oder widerrufen werden. Die Seite zeigt gleich einen passenden Beispielbefehl. Das Token kann alternativ auch ohne Browser geholt werden (Kapitel 4).

 Hausplan

Haushalte

Hilfe & API

 examiner

Abmelden

Profil & API-Zugang

Benutzername

examinator

E-Mail

examinator@example.com

Haushalte

Demo-Wohnung

API-Token

Das Token ersetzt beim API das Passwort. Du kannst es auch ohne Browser holen: `http --auth examiner POST https://hausplan.nusnet.ch/api/tokens`

Dein Token (gültig bis 16.10.2026 20:47 UTC):

c9ac73f7879131470417d0227d8b6cb024268a1432ad6cb0e4c994bd1119bfe8

http GET https://hausplan.nusnet.ch/api/households "Authorization:Bearer c9ac73f7879131470417d0227d8b6cb024268a1432ad6cb0e4c994bd1119bfe8"

Token anzeigen / erstellen

Token widerrufen

HausPlan · Praxisarbeit DBWE · Flask & PostgreSQL

4 REST-API

4.1 Zugriff und Authentifizierung

Das API ist unter **https://hausplan.nuscnet.ch/api/** erreichbar (HTTPS, Port 443) und liefert JSON. Die Authentifizierung erfolgt ohne Browser in zwei Schritten, umgesetzt mit Flask-HTTPAuth nach Grinberg (2024, Kap. 23):

1. Mit Benutzername und Passwort (HTTP Basic Auth) wird über POST /api/tokens ein Token geholt. Es ist 7 Tage gültig.
2. Bei allen weiteren Anfragen wird das Token im Header Authorization: Bearer <token> mitgesendet. Mit DELETE /api/tokens kann es widerrufen werden.

```
http --auth examiner:Okto2026 POST https://hausplan.nuscnet.ch/api/tokens
curl -u examiner:Okto2026 -X POST https://hausplan.nuscnet.ch/api/tokens

{"expires":"2026-10-16T20:47:16.223270+00:00","token":"338d6795...f0fc0ed2"}
```

4.2 Endpunkte

Alle Endpunkte ausser POST /api/tokens verlangen das Token. Benutzer sehen nur Haushalte, in denen sie Mitglied sind. Für die Demo-Daten gilt: Haushalt 1 = «Demo-Wohnung», Haushalt 2 = «WG des Bruders» (kein Zugriff für examiner).

Methode	Endpunkt-URL	Beschreibung
POST	https://hausplan.nuscnet.ch/api/tokens	Token holen (Basic Auth)
DELETE	https://hausplan.nuscnet.ch/api/tokens	Eigenes Token widerrufen
GET	https://hausplan.nuscnet.ch/api/users/me	Eigenes Profil mit Haushalten
GET	https://hausplan.nuscnet.ch/api/households	Haushalte, in denen ich Mitglied bin
GET	https://hausplan.nuscnet.ch/api/households/1	Haushalt mit Mitgliedern, Rollen, Kriterien, Budget
GET	https://hausplan.nuscnet.ch/api/households/1/projects	Alle Projekte; Filter ?status=geplant (idee, bewertet, geplant, in_arbeit, erledigt, verworfen)
POST	https://hausplan.nuscnet.ch/api/households/1/projects	Neue Idee erfassen (JSON: title, description, estimated_cost, estimated_hours)
GET	https://hausplan.nuscnet.ch/api/households/1/ranking	Offene Projekte nach Score sortiert
GET	https://hausplan.nuscnet.ch/api/households/1/order	Empfohlene Reihenfolge
GET	https://hausplan.nuscnet.ch/api/households/1/plan	Budget-Vorschlag «Was liegt drin?»
GET	https://hausplan.nuscnet.ch/api/households/1/stats	Projekte pro Status, Schätzgenauigkeit
GET	https://hausplan.nuscnet.ch/api/projects/4	Projekt mit Einzelbewertungen, Konsens und Abhängigkeiten

Tabelle 5: Endpunkte des REST-API

4.3 Beispiele

Rangliste der offenen Projekte (Auszug):

```
http GET https://hausplan.nuscnet.ch/api/households/1/ranking "Authorization:Bearer $T"

{"items": [{"rank": 1, "id": 5, "title": "Heizungssteuerung smart machen",
  "status": "in_arbeit", "score": 62.5, "disagreement": true,
  "estimated_cost": 450.0, "estimated_hours": 10.0, "blocked": false,
  "_links": {"self": "/api/projects/5"}}, ...]}
```

Neue Idee erfassen (schreibender Zugriff, Antwort 201 Created):

```
http POST https://hausplan.nuscnet.ch/api/households/1/projects "Authorization:Bearer $T" \
  title="Pflanzen umtopfen" estimated_cost:=30 estimated_hours:=1
```

Kennzahlen und Schätzgenauigkeit:

```
curl https://hausplan.nuscnet.ch/api/households/1/stats -H "Authorization: Bearer $T"

{"estimation": {"completed_projects": 2, "cost_deviation_percent": 7.1,
  "hours_deviation_percent": 31.2, "messages": ["Kosten: Ihr habt im Schnitt um 7 %
  unterschätzt.",
  "Zeit: Ihr habt im Schnitt um 31 % unterschätzt."]},
  "projects_by_status": {"bewertet": 3, "erledigt": 2, "geplant": 1, "idee": 1, "in_arbeit": 1,
  "verworfen": 1}}
```

4.4 Fehlercodes

Code	Bedeutung	Beispielantwort
400	Ungültige Eingabe (fehlender Titel, negativer Betrag, unbekannter Status)	{"error": "Bad Request", "message": "Feld 'title' ist erforderlich."}
401	Token fehlt, ist ungültig oder abgelaufen; falsches Passwort	{"error": "Unauthorized", "message": "Token fehlt, ist ungültig oder abgelaufen."}
403	Kein Mitglied dieses Haushalts	{"error": "Forbidden", "message": "Kein Zugriff auf diese Ressource."}
404	Ressource existiert nicht	{"error": "Not Found"}

Tabelle 6: Fehlercodes des API

5 Architektur

5.1 Überblick und Code-Struktur

HausPlan ist eine server-gerenderte Flask-Anwendung. Der Aufbau folgt dem Flask Mega-Tutorial aus dem Unterricht (Grinberg, 2024): Application Factory, Konfiguration über Umgebungsvariablen, Blueprints für Authentifizierung, Weboberfläche und API, Flask-SQLAlchemy mit Flask-Migrate für die Datenbank, Flask-Login und Flask-WTF für Anmeldung und Formulare, Flask-HTTPAuth für das API.

Die wichtigste Abweichung vom Unterricht ist das Paket `app/services/`: Die gesamte Geschäftslogik liegt dort und ist unabhängig von HTTP. Web-Routen und API-Endpunkte rufen dieselben Funktionen auf. Damit gelten die Regeln überall gleich und lassen sich ohne Webserver direkt testen.

```
hausplan/
├─ hausplan.py      Einstiegspunkt (Flask-CLI, Gunicorn: hausplan:app)
├─ config.py        Konfiguration aus Umgebungsvariablen (.env)
├─ app/
│  ├─ __init__.py   Application Factory, Erweiterungen, Blueprints, ProxyFix
│  ├─ models.py     Datenmodell (SQLAlchemy 2)
│  ├─ services/     Geschäftslogik: scoring, workflow, dependencies,
│  │               planning, ratings, access
│  ├─ auth/         Blueprint Registrierung / Login
│  ├─ main/         Blueprint Weboberfläche (Flask-WTF)
│  ├─ api/          Blueprint REST-API (Flask-HTTPAuth)
│  ├─ templates/    Jinja2-Templates (Bootstrap 5, lokal ausgeliefert)
│  └─ cli.py        flask seed / flask create-user
├─ migrations/     Alembic-Migrationen (Flask-Migrate)
├─ tests/          30 automatisierte Tests (pytest)
└─ deploy/         systemd, Nginx, cloudflared, Setup- und Backup-Skripte
```

5.2 Datenmodell

Die folgende Abbildung zeigt das Datenmodell als ERD. Es besteht aus sieben Tabellen.

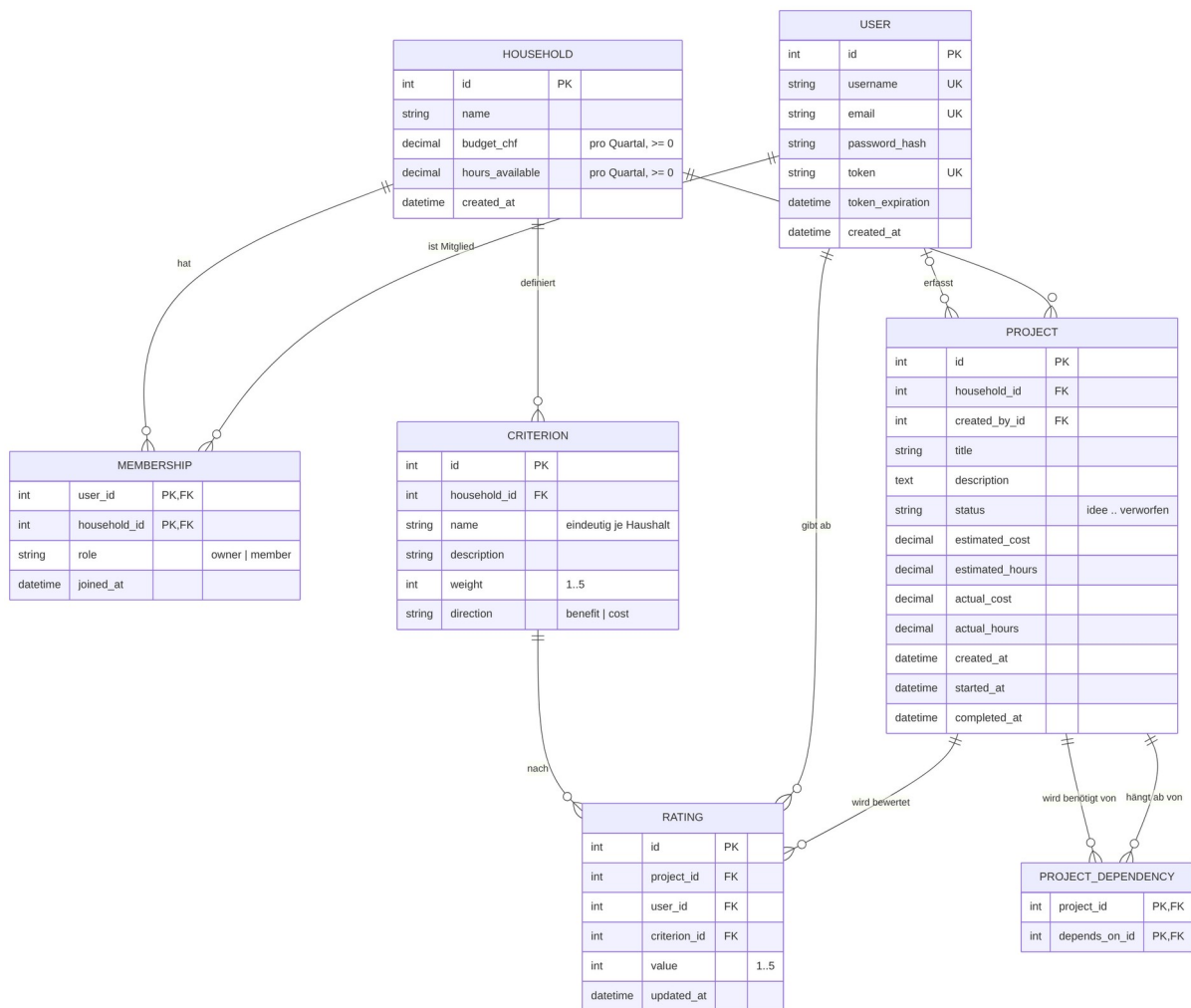


Abbildung 8: Datenmodell (ERD)

Tabelle	Beschreibung
user	Benutzerkonto mit eindeutigem Benutzernamen und E-Mail, Passwort-Hash sowie API-Token mit Ablaufzeit.
household	Haushalt mit Planungsrahmen (Budget in CHF und verfügbare Stunden pro Quartal).
membership	Assoziationstabelle der n:m-Beziehung User ↔ Household mit dem Zusatzattribut role (owner/member).
criterion	Bewertungskriterium eines Haushalts mit Gewicht 1–5 und Richtung (benefit/cost); Name eindeutig pro Haushalt.
project	Projekt eines Haushalts (1:n) mit Status, geschätzten und effektiven Kosten und Stunden sowie Zeitstempeln.
rating	Bewertung einer Person für ein Projekt nach einem Kriterium (Wert 1–5). Unique-Constraint verhindert Doppelbewertungen.
project_dependency	n:m-Selbstbeziehung von project: «project_id hängt ab von depends_on_id» (Muster «Followers» aus dem Unterricht).

Tabelle 7: Tabellen des Datenmodells

Check-Constraints in PostgreSQL sichern die Integrität zusätzlich zur Validierung in Python ab (Wertebereiche, nicht-negative Beträge, gültige Status). Dass die Abhängigkeiten zyklensfrei bleiben, kann die Datenbank nicht prüfen; diese Regel übernimmt die Geschäftslogik.

5.3 Geschäftslogik

Gewichtete Nutzwertanalyse (scoring.py)

Jedes Mitglied bewertet pro Kriterium k einen Wert v_k von 1 bis 5. Bei Kosten-Kriterien (z. B. Aufwand) ist ein hoher Wert schlecht und wird umgekehrt. Der persönliche Score ist der gewichtete Mittelwert, linear auf 0–100 umgerechnet (Methode nach Zangemeister, 1976):

$$\begin{aligned} \text{Teilnutzen}_k &= v_k \text{ (Nutzen)} \quad \text{bzw.} \quad 6 - v_k \text{ (Kosten)} \\ \text{Mittel} &= \sum (g_k \cdot \text{Teilnutzen}_k) / \sum g_k & \text{Score} &= (\text{Mittel} - 1) / 4 \cdot 100 \end{aligned}$$

Der Projekt-Score ist der Durchschnitt aller Personen, die vollständig bewertet haben. **Konsens-Erkennung:** Ein Projekt erhält «Diskussionsbedarf», wenn die persönlichen Scores mehr als 25 Punkte auseinanderliegen oder bei einem Kriterium die Bewertungen um mindestens 3 Stufen abweichen.

Status-Workflow (workflow.py)

Der Lebenszyklus eines Projekts ist als Zustandsautomat mit Bedingungen (Guards) umgesetzt, siehe Kapitel 5.4. Die Oberfläche zeigt für jeden gesperrten Übergang den Grund.

Abhängigkeiten und Reihenfolge (dependencies.py)

Abhängigkeiten bilden einen gerichteten Graphen. Bevor eine neue Kante gespeichert wird, prüft eine Tiefensuche, ob sie einen Zyklus schliessen würde (Cormen et al., 2022); in diesem Fall wird sie mit der ganzen Kette abgelehnt, z. B. «Wand streichen → Lampe montieren → Wand streichen». Die empfohlene Reihenfolge entsteht durch topologische Sortierung nach Kahn (1962); stehen mehrere Projekte zur Wahl, entscheidet eine Prioritätswarteschlange nach Score.

Budget-Vorschlag und Schätzgenauigkeit (planning.py)

Die Auswahl der Projekte für das Quartal ist ein mehrdimensionales Rucksackproblem. Für die kleinen Mengen eines Haushalts genügt eine Greedy-Heuristik (Dantzig, 1957): Es wird wiederholt das Projekt mit dem höchsten Nutzen pro Ressourcenanteil gewählt, das noch in Budget und Zeit passt und dessen Abhängigkeiten erfüllt sind. Laufende Projekte reservieren vorab Budget und Zeit.

$$\text{Effizienz} = \text{Score} / (\text{Kosten} / \text{Budget} + \text{Stunden} / \text{Stunden_verfügbar})$$

Die Schätzgenauigkeit ist die mittlere prozentuale Abweichung der effektiven von den geschätzten Kosten bzw. Stunden über alle abgeschlossenen Projekte.

5.4 Wichtigste Abläufe

Zustandsdiagramm: Projekt-Workflow

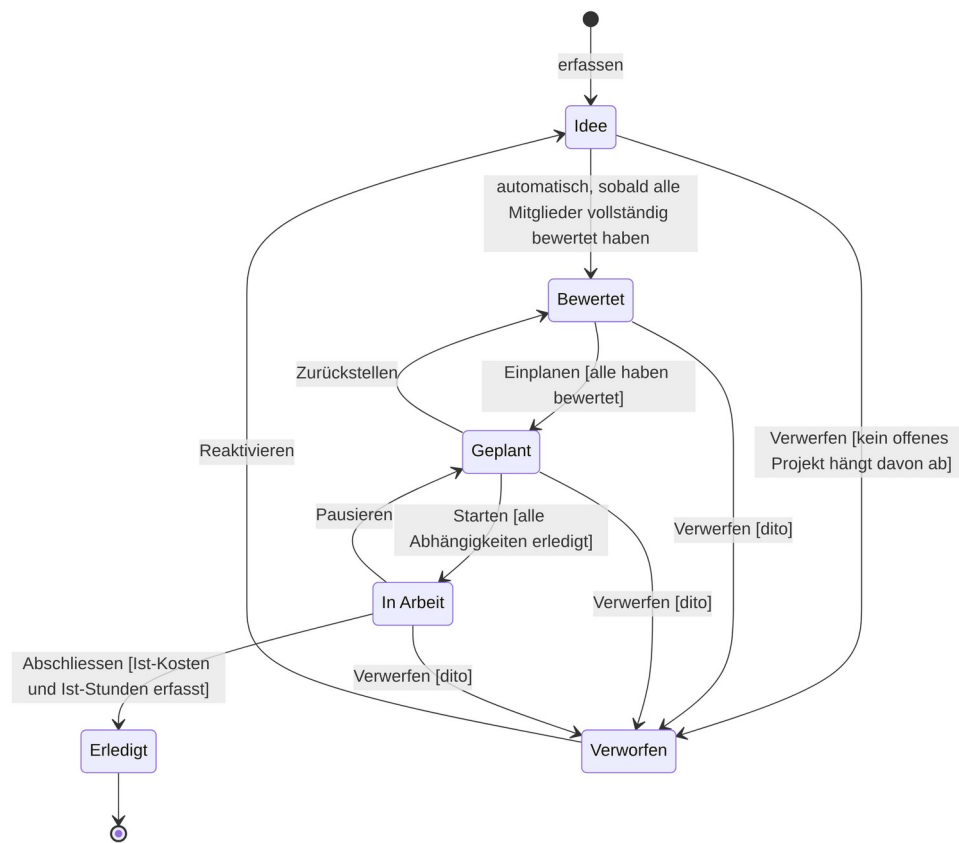


Abbildung 9: Zustandsdiagramm des Projekt-Workflows

Ein Projekt startet als «Idee». Sobald alle Mitglieder vollständig bewertet haben, wechselt es automatisch auf «Bewertet». Eingeplant werden kann nur, wenn alle bewertet haben; gestartet nur, wenn alle Abhängigkeiten erledigt sind; abgeschlossen nur mit Ist-Kosten und Ist-Stunden. Verworfen werden kann ein Projekt nur, wenn kein offenes Projekt davon abhängt. Verworfen Projekte lassen sich reaktivieren.

Sequenzdiagramm: Bewertung abgeben

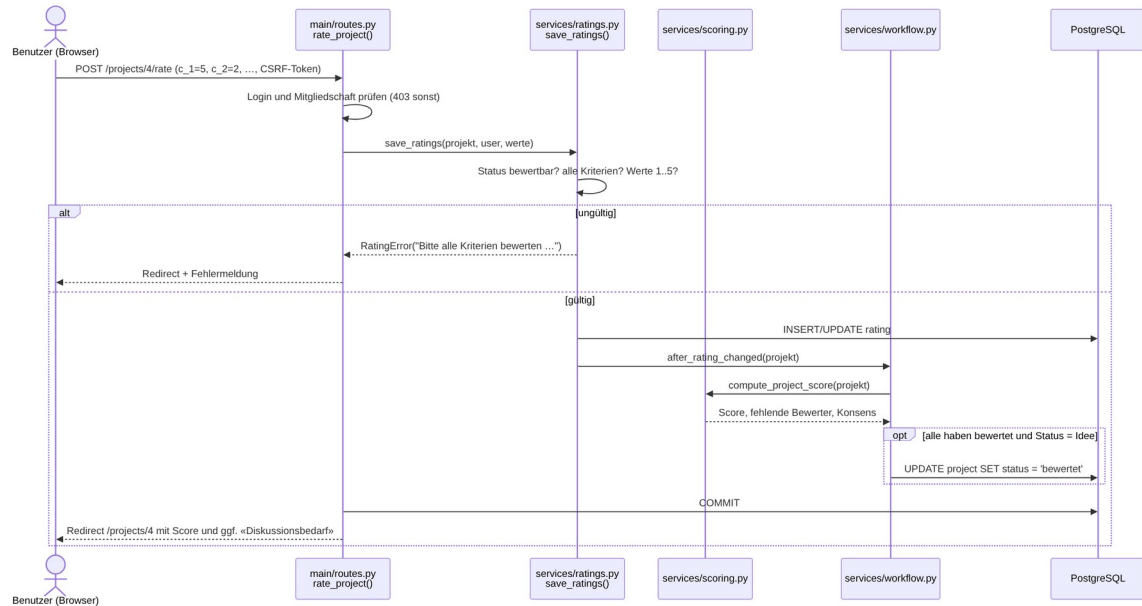


Abbildung 10: Sequenzdiagramm «Bewertung abgeben»

Die Route prüft Anmeldung und Mitgliedschaft und übergibt die Werte an `save_ratings()`. Diese prüft Status, Vollständigkeit und Wertebereich, speichert die Bewertung und ruft den Workflow auf, der über die Score-Berechnung entscheidet, ob das Projekt automatisch auf «Bewertet» wechselt. Erst danach wird die Transaktion bestätigt.

Aktivitätsdiagramm: Budget-Vorschlag

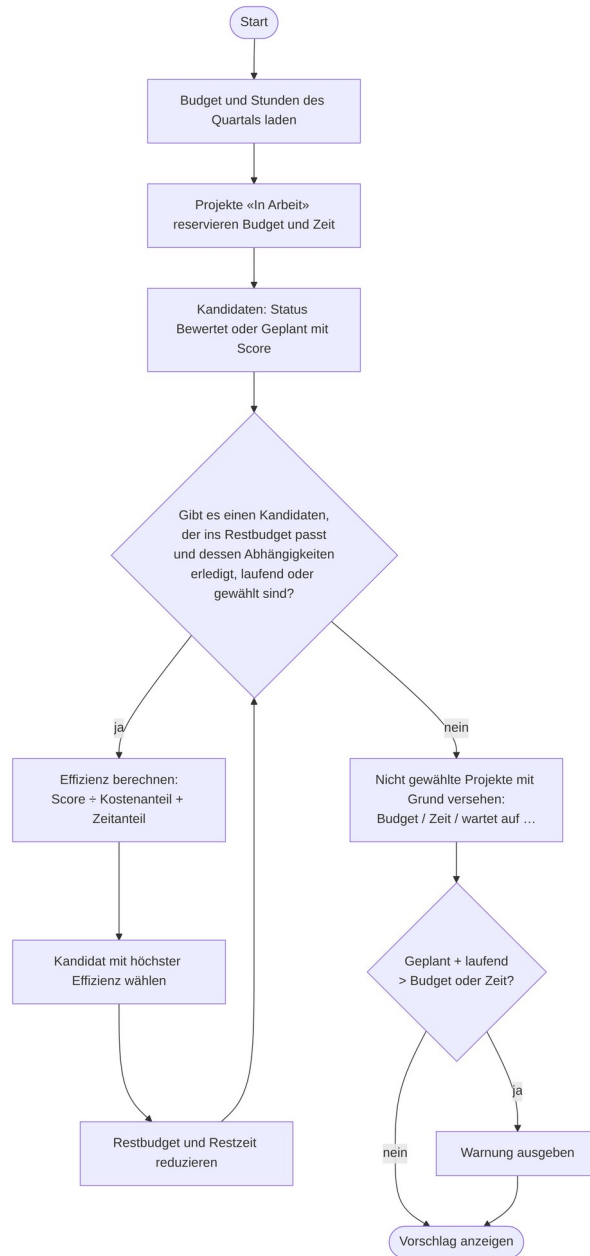


Abbildung 11: Aktivitätsdiagramm «Was liegt drin?»

Der Ablauf reserviert zuerst Budget und Zeit für laufende Projekte und wählt dann schrittweise das effizienteste passende Projekt, bis keines mehr passt. Nicht gewählte Projekte erhalten eine Begründung, und bei Überschreitung durch bereits geplante Projekte wird gewarnt.

5.5 Zusätzliche Technologien

Die folgenden Technologien wurden nicht im Unterricht behandelt und werden hier begründet:

Technologie	Einsatz und Begründung	Quelle
Proxmox VE (LXC)	Virtualisierung auf vorhandener Heiminfrastruktur. App und Datenbank laufen in getrennten Containern, die einzeln gesichert und wiederhergestellt werden können.	Proxmox Server Solutions (o. J.)

Technologie	Einsatz und Begründung	Quelle
Cloudflare Tunnel	Veröffentlichung ohne offenen Port am Router: cloudflared baut eine ausgehende Verbindung auf. Cloudflare liefert DNS, TLS-Zertifikat und DDoS-Schutz; die private IP bleibt verborgen.	Cloudflare (o. J.-a)
Werkzeug ProxyFix, Nginx realip	Hinter Nginx und Tunnel sieht Flask sonst nur 127.0.0.1 und «http». ProxyFix übernimmt Schema und Host, Nginx setzt die echte Besucher-IP aus CF-Connecting-IP.	Pallets (o. J.); Cloudflare (o. J.-b)
pytest	Automatisierte Tests mit frischer In-Memory-Datenbank pro Test.	pytest-dev (o. J.)
Bootstrap 5 (lokal)	Gestaltung. Lokal statt über ein CDN ausgeliefert, damit die App nicht von einem fremden Server abhängt.	Bootstrap Team (o. J.)
Mermaid	Diagramme als Text im Repository, versionierbar.	Mermaid (o. J.)

Tabelle 8: Zusätzliche Technologien

5.6 Bereitstellung

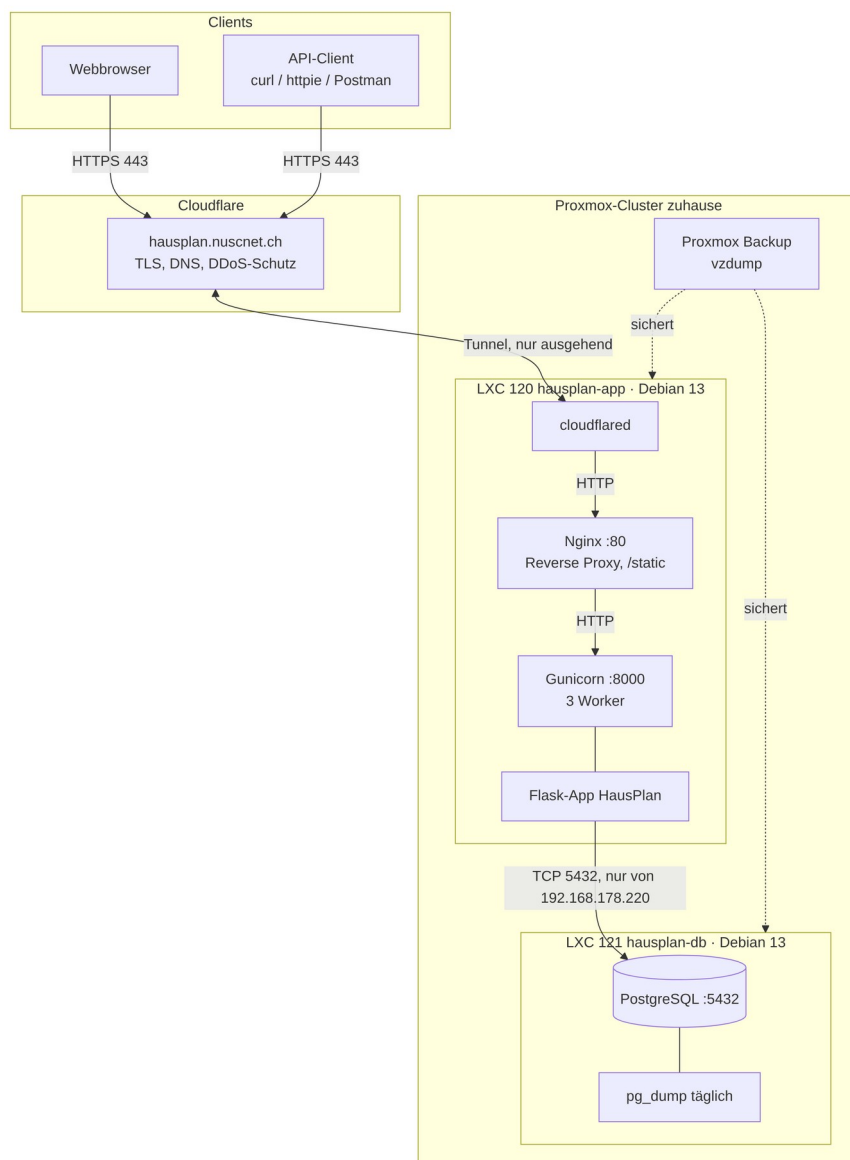


Abbildung 12: Bereitstellungsdiagramm

Komponente	Ort	Aufgabe
Cloudflare Edge	Internet	Öffentlicher Name hausplan.nuscnet.ch, TLS-Zertifikat, Schutz vor Angriffen
cloudflared	LXC 120 hausplan-app (192.168.178.220)	Ausgehender Tunnel zu Cloudflare, leitet an Nginx weiter
Nginx 1.26	LXC 120, Port 80	Reverse Proxy, liefert statische Dateien, setzt echte Client-IP
Gunicorn 26	LXC 120, 127.0.0.1:8000	WSGI-Server mit 3 Worker-Prozessen, systemd-Dienst mit automatischem Neustart
Flask-App	LXC 120, /opt/hausplan	Weboberfläche, API und Geschäftslogik
PostgreSQL 17	LXC 121 hausplan-db (192.168.178.221), Port 5432	Datenbank (UTF-8); nur vom App-Container aus erreichbar (pg_hba.conf)
Backups	Proxmox und LXC 121	Täglicher pg_dump (14 Tage aufbewahrt) und Container-Backups (vzdump)

Tabelle 9: Komponenten der Bereitstellung

Beide Container starten automatisch mit dem Proxmox-Host, die Datenbank zuerst (onboot, startup order). Die Einrichtung ist in Skripten im Ordner `deploy/` festgehalten (`setup_db.sh`, `setup_app.sh`, `update_app.sh`, `backup_db.sh`). Damit das API mit curl oder httpie getestet werden kann, sind in Cloudflare weder Cloudflare Access noch eine Browser-Challenge für die Domain aktiv.

5.7 Sicherheit

- Passwörter werden nur als gesalzener Hash gespeichert (Werkzeug `generate_password_hash`).
- Alle Formulare sind mit CSRF-Token geschützt (Flask-WTF); das API ist davon ausgenommen, da es Tokens statt Cookies verwendet.
- API-Tokens sind zufällig (256 Bit), laufen nach 7 Tagen ab und können widerrufen werden.
- Jede Anfrage auf Haushalte und Projekte prüft die Mitgliedschaft (Web und API gemeinsam in `services/access.py`); Einstellungen dürfen nur Besitzer ändern.
- Session-Cookies sind `HttpOnly`, `SameSite=Lax` und werden nur über HTTPS gesendet; Weiterleitungen nach dem Login akzeptieren nur relative Adressen (Schutz vor Open Redirect).
- Am Router ist kein Port offen; die Datenbank akzeptiert nur Verbindungen vom App-Container mit Passwort (scram-sha-256).

6 Reflexion

6.1 Wartbarkeit

Vorteile: Die Trennung von Geschäftslogik (`services/`) und Oberflächen erlaubt es, Regeln an einer Stelle zu ändern und mit Tests abzusichern. Das Datenbankschema ist mit Alembic versioniert, und das Deployment ist in Skripten reproduzierbar dokumentiert. Die Struktur folgt dem Unterricht, wodurch sich andere Entwickler schnell zurechtfinden. Abhängigkeiten sind in `requirements.txt` auf exakte Versionen festgelegt.

Nachteile: Die server-gerechneten Seiten machen jede Interaktion zu einem vollständigen Seitenaufruf. Mehr Interaktivität würde JavaScript und damit eine zweite Technologie im Frontend erfordern. Die Konfiguration verteilt sich auf zwei Container und Cloudflare, was beim Betrieb Wissen über mehrere Systeme voraussetzt.

6.2 Skalierbarkeit

Vorteile: Die Anwendung ist zustandslos (Sessions im signierten Cookie). Weitere Unicorn-Worker oder zusätzliche App-Container könnten ohne Codeänderung ergänzt werden. Die getrennte Datenbank lässt sich unabhängig vergrößern.

Nachteile: Scores werden bei jedem Aufruf neu berechnet. Für einen Haushalt mit 10–100 Projekten ist das unkritisch, bei vielen Projekten wären gespeicherte Scores oder Caching nötig. Die Greedy-Heuristik liefert nicht immer die optimale Auswahl; für kleine Mengen wäre auch eine exakte Lösung mit dynamischer Programmierung möglich. Die Datenbank bleibt ein einzelner Engpass.

6.3 Verfügbarkeit

Vorteile: Dienste starten nach einem Absturz automatisch neu (`systemd Restart=always`), Container nach einem Neustart des Hosts (`onboot`). Der Cloudflare Tunnel funktioniert auch bei wechselnder öffentlicher IP und ohne Portfreigabe. Tägliche Backups auf zwei Ebenen ermöglichen eine Wiederherstellung.

Nachteile: Strom, Internetanschluss und der einzelne Proxmox-Node sind Single Points of Failure. Der Cluster verfügt zwar über einen gemeinsamen NFS-Speicher, die Container liegen aber bewusst auf lokalem Speicher, weil eine Datenbank über NFS langsamer und fehleranfälliger ist. Damit ist kein automatischer Umzug auf einen anderen Node möglich. Für die Korrekturzeit werden deshalb keine Updates eingespielt und die Erreichbarkeit regelmässig geprüft.

6.4 Erkenntnisse aus der Umsetzung

Die Bereitstellung brachte mehrere Probleme, die lokal nicht sichtbar waren und die Skripte verbessert haben:

- **Zeichensatz der Datenbank:** Das Debian-Template hat keine Spracheinstellung, PostgreSQL legte die Datenbank deshalb mit `SQL_ASCII` an. Umlaute hätten zu Fehlern geführt. Das Skript erstellt die Datenbank jetzt explizit mit UTF-8.
- **Sonderzeichen im Passwort:** Ein `@` im Datenbankpasswort bricht die Verbindungs-URL. Es muss URL-kodiert (`%40`) werden; die Vorlage `.env.example` weist darauf hin.
- **Dateirechte bei Git:** Weil das Repository dem Dienstbenutzer gehört, verweigerte Git als root das Aktualisieren («dubious ownership»). Das Skript führt `git pull` jetzt als Dienstbenutzer aus.
- **Netzwerk-Zugriff unter macOS:** Der Browser erhielt `ERR_ADDRESS_UNREACHABLE`, obwohl der Server antwortete. Ursache war die macOS-Berechtigung «Lokales Netzwerk», nicht die Anwendung.

Die wichtigste Lehre: Ein Testlauf der Installation auf einer frischen Umgebung deckt Fehler auf, die beim Entwickeln nicht auffallen. Die Skripte sind dadurch robuster geworden.

7 Tests

7.1 Teststrategie

Die Geschäftslogik, die Weboberfläche und das API werden mit **30 automatisierten Tests** (pytest) geprüft. Jeder Test erhält eine frische In-Memory-Datenbank. Aufruf: `python -m pytest -v`. Die Bereitstellung im Internet wurde manuell von aussen geprüft. Die Tests sind im Code mit der ID des Testprotokolls benannt (z. B. `test_T08_cycle_rejected`).

7.2 Testprotokoll

Durchgeführt am 09.10.2026. T01–T10 lokal automatisiert, T11–T12 automatisiert gegen das API, M01–M02 manuell über das Internet.

ID	Kurzbeschreibung	Erwartetes Ergebnis	Tatsächliches Ergebnis	Status
T01	Registrierung «nury»; danach «Nury» und gleiche E-Mail erneut; Passwort mit 3 Zeichen	Konto angelegt; Duplikate und zu kurzes Passwort mit Meldung abgelehnt	Wie erwartet	✓
T02	Haushaltsseite ohne Login; Login mit falschem, dann korrektem Passwort	Weiterleitung auf Login; Fehlermeldung; danach Zugriff	Wie erwartet	✓
T03	Bewertung 5/4/3/2 bei Gewichten 3/2/1/2, Aufwand als Kosten-Kriterium; zweite Person 3/3/3/3	Score 81,25; Projekt-Score 65,6 (Mittel aus 81,25 und 50)	81,25 und 65,625	✓
T04	Zwei Personen bewerten «Hochbeet» sehr unterschiedlich bzw. ähnlich	«Diskussionsbedarf» mit Grund «Uneinig bei Nutzen»; bei ähnlichen Werten kein Flag	Wie erwartet	✓
T06	Erste Person bewertet, dann zweite; Sprung Idee → In Arbeit; Abschluss ohne Ist-Werte	Automatischer Wechsel auf «Bewertet»; unerlaubte Übergänge abgelehnt	Wie erwartet	✓
T07	«Lampe» hängt von «Wand» ab: Lampe starten; Wand abschliessen; Lampe erneut starten	Erster Start abgelehnt («Blockiert durch ...»), zweiter erlaubt	Wie erwartet	✓
T08	A → B, B → C anlegen, dann C → A; Selbstabhängigkeit; fremder Haushalt	Zyklus mit Kette abgelehnt, nichts gespeichert	«Zirkuläre Abhängigkeit: C → A → B → C»	✓
T10	Budget 1000 CHF / 20 h, fünf Projekte, «Velo» hängt von «Keller» ab, «Bad» kostet 900 CHF	Summe ≤ Budget und Zeit; Bad mit Grund ausgeschlossen; Velo nur mit Keller	Lampe, Regal, Keller, Velo (650 CHF, 19 h); Bad «Budget reicht nicht»	✓
T11	API: ohne Token; falsches Passwort; Token holen; alle GET-Endpunkte; Token widerrufen	401; 401; Token; alle 200 mit korrekten Daten; danach 401	Wie erwartet	✓
T12	API: Benutzer ohne Mitgliedschaft liest fremden Haushalt und fremdes Projekt; nicht existierendes Projekt	403 als JSON; 403; 404	Wie erwartet	✓
M01	https://hausplan.nuscnet.ch von extern im Browser öffnen und anmelden	Login-Seite über HTTPS, Anmeldung mit Testkonto möglich	Login-Seite über HTTPS erreichbar, Anmeldung erfolgreich	✓

ID	Kurzbeschreibung	Erwartetes Ergebnis	Tatsächliches Ergebnis	Status
M02	Von extern: curl -u examiner:... -X POST https://hausplan.nusnet.ch /api/tokens, danach Anfrage mit ungültigem Token	JSON mit Token, keine Cloudflare-Challenge; ungültiges Token ergibt 401 als JSON	Token erhalten; 401 Unauthorized als JSON	✓

Tabelle 10: Testprotokoll

Zusätzlich automatisiert geprüft (nicht im Protokoll ausgewählt): Schätzgenauigkeit, Schutz des letzten Besitzers, Web-Ablauf «Idee erfassen und bewerten», Zugriffsschutz im Web (403), Filter und Erfassen einer Idee per API inkl. Validierung. Noch nicht durchgeführt ist ein Neustart-Test des Proxmox-Hosts (automatischer Start beider Container); er ist für die Korrekturzeit geplant.

8 Fazit und Ausblick

Alle Anforderungen der Aufgabenstellung sind erfüllt. HausPlan geht über eine reine Datenverwaltung hinaus: Die Nutzwertanalyse mit Konsens-Erkennung, der regelbasierte Workflow, die Abhängigkeiten mit Zyklenerkennung und der Budget-Vorschlag bilden eine eigenständige Geschäftslogik, die durch Tests abgesichert und im API verfügbar ist. Die Bereitstellung auf eigener Infrastruktur über einen Cloudflare Tunnel ist kostengünstig und sicher, hat aber die Heiminfrastuktur als grösstes Risiko.

Mögliche Erweiterungen:

- Fotos vorher/nachher pro Projekt (Objektspeicher statt Datenbank)
- Benachrichtigungen per E-Mail oder Push, wenn eine Bewertung fehlt oder ein Projekt frei wird
- Erfassen von Ideen unterwegs über einen iOS-Kurzbefehl mit dem bestehenden POST-Endpunkt
- Exakte Optimierung des Budget-Vorschlags und Planung über mehrere Quartale
- Hochverfügbarer Betrieb mit gemeinsamem Speicher und externem Monitoring

Quellenverzeichnis

- Bootstrap Team. (o. J.). *Bootstrap 5.3 Documentation*. Abgerufen am 09.10.2026, von <https://getbootstrap.com/docs/5.3/>
- Cloudflare. (o. J.-a). *Cloudflare Tunnel*. Abgerufen am 09.10.2026, von <https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/>
- Cloudflare. (o. J.-b). *Restoring original visitor IPs*. Abgerufen am 09.10.2026, von <https://developers.cloudflare.com/support/troubleshooting/restoring-visitor-ips/restoring-original-visitor-ips/>
- Cormen, T. H., Leiserson, C. E., Rivest, R. L. & Stein, C. (2022). *Introduction to Algorithms* (4. Aufl.). MIT Press.
- Dantzig, G. B. (1957). Discrete-variable extremum problems. *Operations Research*, 5(2), 266–288. <https://doi.org/10.1287/opre.5.2.266>
- Grinberg, M. (2024). *The Flask Mega-Tutorial* (2024 Edition). Abgerufen am 09.10.2026, von <https://blog.miguelgrinberg.com/post/the-flask-mega-tutorial-part-i-hello-world>
- Kahn, A. B. (1962). Topological sorting of large networks. *Communications of the ACM*, 5(11), 558–562. <https://doi.org/10.1145/368996.369025>
- Mermaid. (o. J.). *Mermaid Documentation*. Abgerufen am 09.10.2026, von <https://mermaid.js.org/>
- Pallets. (o. J.). *Flask Documentation: Tell Flask it is Behind a Proxy*. Abgerufen am 09.10.2026, von https://flask.palletsprojects.com/en/stable/deploying/proxy_fix/
- PostgreSQL Global Development Group. (o. J.). *PostgreSQL 17 Documentation*. Abgerufen am 09.10.2026, von <https://www.postgresql.org/docs/17/>
- Proxmox Server Solutions. (o. J.). *Linux Container*. Abgerufen am 09.10.2026, von https://pve.proxmox.com/wiki/Linux_Container
- pytest-dev. (o. J.). *pytest Documentation*. Abgerufen am 09.10.2026, von <https://docs.pytest.org/>
- Zangemeister, C. (1976). *Nutzwertanalyse in der Systemtechnik* (4. Aufl.). Wittemannsche Buchhandlung.

Abbildungsverzeichnis

Abbildung 1: Anmeldeseite.....	7
Abbildung 2: Übersicht «Meine Haushalte».....	8
Abbildung 3: Projekt-Board der Demo-Wohnung.....	8
Abbildung 4: Projektansicht mit Bewertung, Diskussionsbedarf und Status.....	9
Abbildung 5: Planung mit Budget-Vorschlag, Reihenfolge und Schätzgenauigkeit.....	10
Abbildung 6: Einstellungen mit Mitgliedern und Bewertungskriterien.....	11
Abbildung 7: Profil mit API-Token.....	11
Abbildung 8: Datenmodell (ERD).....	15
Abbildung 9: Zustandsdiagramm des Projekt-Workflows.....	17
Abbildung 10: Sequenzdiagramm «Bewertung abgeben».....	18
Abbildung 11: Aktivitätsdiagramm «Was liegt drin?».....	19
Abbildung 12: Bereitstellungsdiagramm.....	20

Anhang

A Zugriffsinformationen

Webanwendung	https://hausplan.nuscn.ch (HTTPS, Port 443)
API	https://hausplan.nuscn.ch/api/... (HTTPS, Port 443), Authentifizierung siehe Kapitel 4.1
Testkonten	examinator / Okto2026 und mitbewohner / Okto2026
Quellcode	https://github.com/ulangtaun/hausplan (öffentlich lesbar)
Weitere Dokumentation im Repository	README.md, docs/architektur.md, docs/api.md, docs/benutzerhandbuch.md, docs/testprotokoll.md, docs/deployment.md

B Installation und lokaler Start

```
git clone https://github.com/ulangtaun/hausplan.git && cd hausplan
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
flask db upgrade                # ohne .env wird SQLite verwendet
flask seed --password 'Okto2026' # Demo-Daten und Testkonten
flask run                       # http://127.0.0.1:5000
python -m pytest -v             # 30 automatisierte Tests
```

Die Einrichtung auf Proxmox (Container, PostgreSQL, Gunicorn, Nginx, Cloudflare Tunnel, Backups) ist Schritt für Schritt in docs/deployment.md beschrieben.

C Eingesetzte Hilfsmittel

Bei Ideenfindung, Programmierung und Dokumentation wurde der KI-Assistent Claude (Anthropic) als Hilfsmittel eingesetzt. Konzept, Entscheidungen, Bereitstellung, Tests und die Prüfung aller Inhalte lagen beim Autor. Code aus fremden Quellen ist im Quellcode mit Quellenangabe gekennzeichnet.